Assignment-12.5

Name:k.likhith
RollNo:2303A51331
Batch-no:20
Task1: Sorting – Merge Sort Implementation

```python
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr)//2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
nums = list(map(int, input("Enter numbers separated by space: ").split()))
sorted_nums = merge_sort(nums)
print("Sorted List:", sorted nums)
```

Output:

```
Enter numbers separated by space: 2
Sorted List: [2]
```

Observation: The list was successfully sorted using the divide-and-conquer approach. The algorithm maintained O(n log n) time complexity for all cases.

Task2: Searching – Binary Search with AI loptimization

```python
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high)//2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
arr = list(map(int, input("Enter sorted numbers: ").split()))
target = int(input("Enter element to search: "))
index = binary_search(arr, target)
if index != -1:
    print("Element found at index:", index)
else:
    print("Element not found")
```

output:

```
Enter sorted numbers: 1 2 3 4 5
Enter element to search: 3
Element found at index: 2
```

Observation: The target element was found efficiently in a sorted list.
The algorithm reduced search time to O(log n).

Task3: Smart Healthcare Appointment Scheduling

```python
appointments = []
n = int(input("Enter number of appointments: "))
for _ in range(n):
    id = int(input("Appointment ID: "))
    patient = input("Patient Name: ")
    doctor = input("Doctor Name: ")
    time = int(input("Appointment Time: "))
    fee = int(input("Consultation Fee: "))
    appointments.append({"id": id, "patient": patient,
                         "doctor": doctor, "time": time, "fee": fee})
choice = input("Sort by (time/fee): ")
appointments.sort(key=lambda x: x[choice])
print("Sorted Appointments:", appointments)
search_id = int(input("Enter ID to search: "))
appointments.sort(key=lambda x: x["id"])
ids = [x["id"] for x in appointments]
index = binary_search(ids, search_id)
if index != -1:
    print("Found:", appointments[index])
else:
    print("Not Found")
```

Output:

```
Enter number of appointments: 1
Appointment ID: 120
Patient Name: giri
Doctor Name: likhith
Appointment Time: 9
Consultation Fee: 1200
Sort by (time/fee): time
Sorted Appointments: [{'id': 120, 'patient': 'giri', 'doctor': 'likhith', 'time': 9, 'fee': 1200}]
Enter ID to search: 120
```

Observation: Appointment records were sorted correctly based on selected fields. Searching by ID was fast and accurate using Binary Search.

Task4: Railway Ticket Reservation System

```python
tickets = []
n = int(input("Enter number of tickets: "))
for _ in range(n):
    id = int(input("Ticket ID: "))
    name = input("Passenger Name: ")
    train = int(input("Train Number: "))
    seat = int(input("Seat Number: "))
    date = int(input("Travel Date (YYYYMMDD): "))

    tickets.append({"id": id, "name": name,
                    "train": train, "seat": seat, "date": date})
choice = input("Sort by (date/seat): ")
tickets.sort(key=lambda x: x[choice])
print("Sorted Tickets:", tickets)
search_id = int(input("Enter Ticket ID to search: "))
tickets.sort(key=lambda x: x["id"])
ids = [x["id"] for x in tickets]
index = binary_search(ids, search_id)
if index != -1:
    print("Found:", tickets[index])
else:
    print("Not Found")
```

Output:

```
Enter number of tickets: 3
Ticket ID: 1234
Passenger Name: name
Train Number: 321227
Seat Number: 3
Travel Date (YYYYMMDD): 20230323
Ticket ID: 12345
Passenger Name: likhith
Train Number: 21345
Seat Number: 5
Travel Date (YYYYMMDD): 20230724
Ticket ID: 1267
Passenger Name: giri
Train Number: 3423
Seat Number: 13
Travel Date (YYYYMMDD): 20230824
Sort by (date/seat): date
Sorted Tickets: [{'id': 1234, 'name': 'name', 'train': 321227, 'seat': 3, 'date': 20230323}, {'id': 12345, 'name': 'likhith', 'train'
: 21345, 'seat': 5, 'date': 20230724}, {'id': 1267, 'name': 'giri', 'train': 3423, 'seat': 13, 'date': 20230824}]
```

Observation: Ticket data was sorted properly by date or seat number.
Binary Search improved ticket lookup efficiency.

Task5: Smart Hostel Room Allocation System

```python
rooms = []

n = int(input("Enter number of records: "))

for _ in range(n):
    student_id = int(input("Student ID: "))
    room = int(input("Room Number: "))
    floor = int(input("Floor: "))
    date = int(input("Allocation Date (YYYYMMDD): "))

    rooms.append({"student_id": student_id,
                  "room": room,
                  "floor": floor,
                  "date": date})

choice = input("Sort by (room/date): ")

rooms.sort(key=lambda x: x[choice])
print("Sorted Records:", rooms)
```

Output:

```
Enter number of records: 3
Student ID: 101
Room Number: 120
Floor: 1
Allocation Date (YYYYMMDD): 20230409
Student ID: 102
Room Number: 108
Floor: 1
Allocation Date (YYYYMMDD): 20230708
Student ID: 103
Room Number: 428
Floor: 4
Allocation Date (YYYYMMDD): 20230603
Sort by (room/date): room
Sorted Records: [{'student_id': 102, 'room': 108, 'floor': 1, 'date': 20230708}, {'student_id': 101, 'room': 120, 'floor': 1, 'date':
 20230409}, {'student_id': 103, 'room': 428, 'floor': 4, 'date': 20230603}]
```

Observation: Room records were organized correctly after sorting.
Student ID search was efficient and accurate.

Task6: Online Movie Streaming Platform

```python
movies = []

n = int(input("Enter number of movies: "))

for _ in range(n):
    id = int(input("Movie ID: "))
    title = input("Title: ")
    genre = input("Genre: ")
    rating = float(input("Rating: "))
    year = int(input("Release Year: "))

    movies.append({"id": id, "title": title,
                   "genre": genre,
                   "rating": rating,
                   "year": year})

choice = input("Sort by (rating/year): ")

movies.sort(key=lambda x: x[choice])
print("Sorted Movies:", movies)
```

Output:

```
Enter number of movies: 2
Movie ID: 1234
Title: rajasaab
Genre: horror comedy
Rating: 2
Release Year: 2025
Rating: 2
Release Year: 2025
Movie ID: 1345
Title: hanuman
Title: hanuman
Genre: fantasy
Rating: 9.3
Release Year: 2024
Sort by (rating/year): year
Sorted Movies: [{'id': 1345, 'title': 'hanuman', 'genre': 'fantasy', 'rating': 9.3, 'year': 2024}, {'id': 1234, 'title': 'rajasaab'
'genre': 'horror comedy', 'rating': 2.0, 'year': 2025}]
```

Observation : Movies were sorted successfully by rating or release year.
Searching by movie ID worked quickly on sorted data**.**

Task7: Smart Agriculture Crop Monitoring System

```python
crops = []

n = int(input("Enter number of crops: "))

for _ in range(n):
    id = int(input("Crop ID: "))
    name = input("Crop Name: ")
    moisture = int(input("Soil Moisture Level: "))
    yield_est = int(input("Yield Estimate: "))

    crops.append({"id": id, "name": name,
                  "moisture": moisture,
                  "yield": yield_est})

choice = input("Sort by (moisture/yield): ")

crops.sort(key=lambda x: x[choice])
print("Sorted Crops:", crops)
```

Output:

```
Enter number of crops: 2
Crop ID: 1234
Crop Name: rabi
Soil Moisture Level: 3
Yield Estimate: 3
Crop ID: 12344
Crop Name: wheat
Soil Moisture Level: 5
Yield Estimate: 6
Sort by (moisture/yield): moisture
Sorted Crops: [{'id': 1234, 'name': 'rabi', 'moisture': 3, 'yield': 3}, {'id': 12344, 'name': 'wheat', 'moisture': 5, 'yield': 6}]
```

Observation: Crop data was sorted correctly based on moisture or yield.
Crop search by ID was efficient using Binary Search.

Task8: Airport Flight Management System

```python
flights = []

n = int(input("Enter number of flights: "))

for _ in range(n):
    id = int(input("Flight ID: "))
    airline = input("Airline Name: ")
    departure = int(input("Departure Time: "))
    arrival = int(input("Arrival Time: "))
    status = input("Status: ")

    flights.append({"id": id,
                    "airline": airline,
                    "departure": departure,
                    "arrival": arrival,
                    "status": status})

choice = input("Sort by (departure/arrival): ")

flights.sort(key=lambda x: x[choice])
print("Sorted Flights:", flights)
```

Output:

```
Enter number of flights: 2
Flight ID: 1234
Airline Name: ram
Departure Time: 5
Airline Name: ram
Departure Time: 5
Arrival Time: 9
Departure Time: 5
Arrival Time: 9
Arrival Time: 9
Status: safe
Flight ID: 234
Flight ID: 234
Airline Name: era
Departure Time: 9
Arrival Time: 3
Status: safe
Sort by (departure/arrival): arrival
Sorted Flights: [{'id': 234, 'airline': 'era', 'departure': 9, 'arrival': 3, 'status': 'safe'}, {'id': 1234, 'airline': 'ram', 'depar
ture': 5, 'arrival': 9, 'status': 'safe'}]
```

Observation: Flight records were arranged correctly by departure or arrival time. Flight ID search was fast and reliable.